Syllabus-Grounded Contextual Biasing for Code-Switched Lecture ASR

Research Implementation Plan (Methodology)

Final-Year BTech Project | Information Technology | 2026–27

Version 1.0

---

1. Purpose and Scope of This Document

This document specifies **what to build, what to measure, and in what order** for a research study on improving automatic speech recognition (ASR) of code-switched technical lectures using syllabus knowledge. It is a methodology specification, not a code listing: every step is defined at the level of inputs, outputs, decisions and acceptance criteria, so that implementation can proceed independently.

In scope. Data preparation, ASR baseline establishment, syllabus knowledge-base construction, retrieval, three distinct grounding mechanisms, a confidence-gated variant, evaluation protocol, statistical testing, and the mapping from experiments to paper sections.

Out of scope. Downstream note generation and summarisation from the corrected transcript. That is a separate stage of the wider project and is deliberately excluded here so that the ASR contribution can be evaluated in isolation.

Deliverable. A research paper reporting a controlled comparison of syllabus grounding mechanisms, with the transcripts produced by the best system handed to the note-generation stage.

---

2. Research Framing

2.1 Problem

General-purpose ASR models are trained on broad, general-domain speech. In a technical classroom they fail in a specific and predictable way: domain terminology is under-represented in training, so the decoder substitutes acoustically similar but lexically common alternatives. A term such as *matplotlib* is decoded as three ordinary words; *NumPy* becomes a common noun. These are low-frequency errors by word count but high-impact by meaning, because the mistranscribed tokens are exactly the content-bearing terms a student needs.

The problem compounds in Indian classrooms, where instruction is code-switched: a Hindi matrix language carrying English technical terms. The model must simultaneously handle language mixing and specialised vocabulary.

2.2 Observation motivating the work

In an educational deployment, the correct vocabulary is *already known in advance*. Every course has a syllabus. The lecture that is being transcribed is a lecture on a known topic, from a known unit, with a known set of terms. This knowledge is available before the audio is ever recorded, yet a general ASR system makes no use of it.

2.3 Gap

Contextual biasing of ASR toward a known vocabulary is an established research area. What has not been systematically studied is:

1. whether a **course syllabus**, as an authentic and freely available artefact, is an effective biasing source;
2. **at which point in the ASR pipeline** that knowledge is best injected, when several mechanically distinct injection points are available in the same model; and
3. how these mechanisms behave specifically on **Hindi–English code-switched lecture speech**, where the biasing terms are predominantly English tokens embedded in a Hindi matrix.

This study addresses all three.

2.4 Hypotheses

H1. Injecting syllabus-derived context into the ASR pipeline reduces word error rate on code-switched technical lecture speech relative to an unmodified baseline.

H2. The reduction is *concentrated* in domain terminology rather than distributed uniformly; corpus-level WER understates the effect, and a term-restricted metric is required to observe it.

H3. The injection mechanisms differ materially in effectiveness. Specifically, token-level biasing and output-level correction target different error types and are at least partially complementary.

H4. Applying grounding globally causes over-biasing: terms are hallucinated into utterances that did not contain them.

Restricting grounding to regions where the model is *already uncertain* retains most of the terminology gain while substantially reducing this penalty.

H4 is the study's principal claim and the source of its methodological contribution.

2.5 Stated contributions

1. A systematic comparison of three contextual-biasing injection points, driven by a single syllabus knowledge source, on Hindi–English code-switched lecture speech.
2. A confidence-gated biasing strategy that reduces the over-biasing penalty, evaluated against globally applied biasing.
3. A decomposition of code-switched WER into orthographic and acoustic components, quantifying how much apparent error is script-convention mismatch rather than recognition failure.

Framing note. The paper must not claim novelty for prompting or contextual biasing as techniques. It claims novelty for the comparison, the gating strategy, and the domain. Overclaiming on a known technique is the most common reason such papers are rejected or challenged in a viva; a well-controlled comparison with honest negative results is defensible.

---

3. Data

3.1 Corpus

OpenSLR SLR104, Hindi–English subset, from the MUCS 2021 sub-task 2 challenge.

Properties relevant to this study:

- Extracted from spoken tutorials on technical topics. Code-switching arises predominantly from the technical content of the lectures, which makes it a near-ideal proxy for classroom lecture speech.
- Train portion 89.86 hours; test portion 5.18 hours; a separate blind test set exists.
- Audio is 16 kHz, 16-bit. No resampling is required.
- Vocabulary size 17,877 types for the Hindi–English pair.
- Distributed in Kaldi convention: a transcript file, an audio index, a speaker map, and a segments file giving sentence-level timestamps used to cut utterances from the long tutorial recordings.

3.2 Splits and download policy

Download the **test tarball only** (approximately 443 MB). The train tarball is approximately 7.3 GB and is required only if terminology is to be mined from in-domain training transcripts. No fine-tuning is performed in this study, so train data is optional.

If train data is used at any point, it must be used **only** for building the term lexicon, and the report must state explicitly that train and test are disjoint. Mining terminology from test transcripts is test-set leakage and invalidates the study.

3.3 Three-tier evaluation strategy

Running every condition on the full test set is not affordable on a laptop. Define three tiers once, freeze them with a recorded random seed, and use them consistently.

Tier	Size	Purpose	Approx. cost per decode
Tier 1 (dev)	~200 utterances	Tuning: thresholds, prompt format, retrieval depth, debugging	~20 min
Tier 2 (eval subset)	~800 utterances	The full experiment matrix and all ablations	~1.2 h
Tier 3 (full test)	5.18 h	Final confirmation, two systems only	~6 h

Tier 1 must be **disjoint** from the utterances used to report results, or all threshold tuning is performed on the test data. The cleanest arrangement: draw Tier 1 and Tier 2 as non-overlapping samples from the test set, tune exclusively on Tier 1, report exclusively on Tier 2 and Tier 3.

The report must state: *"All development and ablation experiments use a fixed random subset of N utterances (seed recorded);*

hyperparameters were selected on a disjoint development slice; final systems are additionally evaluated on the complete Hindi–English test set.”

3.4 Preparation steps

1. Extract the archive and inspect the directory structure.
2. Parse the transcript, audio index and segments files.
3. Cut per-utterance audio once, using the provided timestamps, to mono 16 kHz. Cutting once and reusing the files guarantees every experimental condition sees identical audio.
4. Produce a flat manifest with one record per utterance: identifier, audio path, reference transcript, duration.
5. Sample and freeze the three tiers.

Acceptance criteria. Utterance count and total duration match the published figures for the test set; a random sample of ten cut files plays correctly and matches its reference; no utterance has zero duration.

---

4. System Under Test

4.1 Model

Whisper large-v3-turbo, run locally via the faster-whisper (CTranslate2) implementation. Local execution is required because two of the three grounding mechanisms need decoder-level access and per-token confidence values that hosted APIs do not expose.

Configuration:

- Compute type: 8-bit integer quantisation on CPU (int8). Apple Silicon GPU is not used by this runtime; execution is CPU-bound.
- Beam search decoding with a fixed beam width, temperature zero, so that every run is deterministic and differences between conditions are attributable to the intervention.
- Conditioning on previous text disabled: utterances are independent sentence segments, not a continuous stream.
- Voice-activity filtering disabled: the audio is already sentence-segmented, so this is wasted computation and a source of boundary errors.

4.2 Required pilot: model selection

This step must not be skipped. The turbo variant is optimised for speed and is known to be weaker than large-v3 on non-English audio. Since this study is entirely about non-English, code-switched audio, the choice must be justified empirically rather than assumed.

Run a small pilot on Tier 1 comparing large-v3-turbo against large-v3 under identical settings, and record WER and wall-clock time for each. Three outcomes:

- **Turbo is close in accuracy and much faster.** Proceed with turbo, and report the pilot as justification. This is a legitimate, documented engineering choice.
- **Turbo is substantially worse.** Use large-v3 as the primary system. A paper whose baseline is a knowingly weakened model invites the criticism that the observed gains merely recover what a better baseline already had.
- **Results are mixed.** Report both. Running the full matrix on turbo and confirming the headline result on large-v3 is a strong robustness argument.

Record this decision explicitly in the paper’s experimental setup section.

4.3 Language configuration pilot

Language selection materially affects WER on code-switched audio. Compare, on Tier 1: forcing Hindi, forcing English, and automatic detection. Fix the setting for all subsequent experiments and report which was chosen and why. Do not leave this to the default.

4.4 Caching requirement

Every decode result must be written to a cache keyed by utterance identifier, backend name, and a hash of the full decoding configuration. This is not an optimisation; it is what makes the study feasible:

- A crashed or interrupted run resumes rather than restarts.
- Re-computing metrics on a completed condition costs seconds instead of hours.
- Every output-level method (Section 7.3) and every combination operates on cached text at zero additional decode cost.
- Results remain reproducible after the fact without re-running the model.

---

5. Syllabus Knowledge Resource

5.1 Design principle

The knowledge source must be something a real deployment would plausibly possess: a written syllabus. It must **not** be derived from the evaluation transcripts.

5.2 Construction

Author 8–15 short topic documents in plain text, one per course topic represented in the corpus. The SLR104 tutorials cover technical subjects such as programming languages, scientific computing environments, shell usage and document preparation systems. Each document should read like a syllabus unit: the concepts, commands, function names and jargon that a lecture on that topic would contain, written as natural prose of roughly 200–400 words rather than a bare keyword dump.

Prose matters, for a mechanical reason explained in Section 7.1.

5.3 Two derived artefacts

(a) A retrievable chunk index. Split each document into overlapping passages of a fixed word length, embed each passage with a multilingual sentence-embedding model, and store the vectors alongside passage text and topic label. Multilingual embedding is required because the queries will contain both Devanagari and Latin script.

(b) A term lexicon. Extract candidate technical terms from the syllabus documents: identifier-like tokens, multi-case words, command names, library names. This lexicon serves two roles: it supplies the bias list for the grounding mechanisms, and it defines which words count as “terminology” for the evaluation metrics in Section 8.

Critical constraint. The lexicon defines the metric. If terms are added to the lexicon after observing which words the system gets wrong, the metric is no longer independent of the method. Freeze the lexicon before running any grounded condition, and record its size and construction procedure in the paper.

5.4 Optional in-domain expansion

If terminology coverage proves too thin, mine additional terms from the **train** transcripts only. Report the lexicon size with and without this expansion, and treat coverage as a reported statistic: what fraction of reference word tokens in the evaluation set are lexicon terms? This number sets the ceiling on achievable gain and belongs in the results section.

---

6. Retrieval Architecture

6.1 The circular dependency

To bias decoding toward the right syllabus terms, the system must know the lecture topic. To determine the topic automatically, it needs a transcript. Resolve this with an explicit two-pass design, which is the system’s architecture and warrants a figure in the paper.

6.2 Two-pass pipeline

1. **Pass 1 — unbiased decode.** Transcribe with no grounding. The output is error-prone, particularly on terminology, but is sufficient to identify the subject matter.
2. **Retrieval.** Use the Pass 1 transcript as a query against the syllabus chunk index. Because retrieval is semantic rather than exact-match, a transcript containing *“mat plot lib”* still retrieves the correct topic passage. Take the top-k passages.
3. **Context assembly.** Build a bounded grounding context from the retrieved passages: topic label plus the terminology they contain. The context must respect the model’s context-length constraint, and the truncation policy must be fixed and documented so that all conditions are comparable.
4. **Pass 2 — grounded decode.** Re-transcribe the same audio with the assembled context injected via the mechanism under

test.

5. **Output-level correction.** Optionally repair the Pass 2 output against the retrieved term list.

6.3 Retrieval must be measured separately

Report retrieval accuracy as an intermediate result: how often does the top-1 retrieved topic match the true topic of the utterance? This decomposes end-to-end failure into *retrieval failure* and *biasing failure*, which is the difference between a discussion section that explains results and one that merely reports them.

6.4 Retrieval granularity

Retrieval may operate per-utterance or per-lecture (aggregating all utterances from one recording into a single query). Per-lecture retrieval is more robust, because errors average out across many utterances, and is more realistic — a real system knows which lecture it is processing. Per-utterance retrieval is the harder, more general condition. Choose one as primary, and report the other as an ablation if time permits.

---

7. Grounding Mechanisms

Four conditions, driven by the same retrieved context, differing only in *where* the knowledge enters the pipeline. This is the core experimental variable.

7.1 M1 — Context conditioning

Inject the syllabus context as decoder text context, prepended before transcription begins. This shifts the model's language-model prior over the entire utterance.

Important design detail. Whisper imitates the *style* of the text it is conditioned on. Official guidance for this parameter is that natural prose using the target terms in context outperforms a comma-separated glossary, and that instruction-like text is liable to be transcribed verbatim rather than acted upon. Two consequences:

- Write the injected context as sentences, not as a keyword list.
- Do not phrase it as a command to the model.

The prose-versus-glossary contrast is itself a cheap and informative ablation row.

Expected failure mode. On short or near-silent utterances the model may continue the injected context instead of transcribing the audio. A guard is required (Section 7.5).

7.2 M2 — Token-level biasing

Supply the syllabus terms as hint phrases that raise the decoding probability of the corresponding tokens during beam search. This is mechanically distinct from M1: it acts on the decoder's search, not on its textual context.

Two implementation constraints must be respected:

- Hint phrases and forced-prefix conditioning are mutually exclusive in the runtime; the prefix takes precedence and silently disables the hints. Never set both.
- The number of hint terms supplied is a tunable parameter and interacts strongly with the over-biasing effect. Sweep it on Tier 1.

7.3 M3 — Output-level correction

Operate on the decoded string, with no model access at all. Two variants:

M3a — lexical/phonetic correction. For each output token absent from the term lexicon, find the closest retrieved term under a string-similarity measure and replace it if similarity exceeds a threshold. Deterministic, fast, and easily explained. Sweep the threshold on Tier 1 only.

M3b — constrained model-based correction. Use a language model to repair only misrecognised terminology, under hard constraints: preserve all other words exactly, preserve script, do not translate, punctuate, reorder, add or delete words. Apply a post-check that discards the correction whenever it alters more than a fixed fraction of tokens, and **report the discard rate** — an unconstrained rewrite destroys WER, and demonstrating that you detected and prevented this is a point in the paper's favour.

M3 is worth emphasising as the only mechanism deployable against a hosted ASR API, which is a practical contribution independent of its accuracy.

7.4 G — Confidence-gated biasing (principal contribution)

Every mechanism above is applied globally: the syllabus is pushed at every utterance and every token, including the many that the model already transcribes correctly. This is the source of the over-biasing penalty in H4.

The gating idea. Apply grounding only where the model signals uncertainty.

1. During Pass 1, retain per-segment and per-token confidence scores, available from the runtime alongside word-level timing.
2. Identify low-confidence utterances and low-confidence token spans.
3. Apply the grounding mechanism **selectively**: re-decode only flagged utterances under biasing, and restrict output-level correction to flagged spans.
4. Leave confidently decoded regions entirely untouched.

Testable prediction. Gated grounding achieves most of the terminology gain of global grounding at a materially lower penalty on non-terminology words. This is a curve, not a point: sweep the confidence threshold and plot terminology error against non-terminology error to trace the trade-off frontier.

Secondary variant, if time allows. Script-conditioned biasing. Since the bias terms are overwhelmingly English tokens inside a Hindi matrix, apply biasing only at positions where the decoder is emitting Latin script. Code-switch-aware biasing on code-switched data is a natural fit for the venue and a strong extension paragraph.

7.5 Guards and instrumentation

Guards are not defensive engineering to be hidden; they are measurements to be reported.

- **Context-echo guard.** If the output overlaps the injected context above a threshold, fall back to the unbiased hypothesis. Report the firing rate.
- **Rewrite guard.** For M3b, discard over-aggressive corrections. Report the rate.
- **Regression counter.** For every condition, count utterances whose error increased relative to baseline. This is as important as the mean.

---

8. Evaluation Protocol

8.1 Normalisation — build and validate this before any grounded experiment

On code-switched data, a substantial share of apparent recognition errors are formatting and convention mismatches rather than acoustic failures. If normalisation is not settled first, the baseline WER is inflated, and every subsequent "improvement" partly measures the normaliser rather than the method.

Systematic mismatches to expect and handle:

- **Script choice for English words.** The reference writes technical terms in Latin script; the model sometimes emits the same word transliterated into Devanagari. Same word, scored as an error.
- **Numeral form.** Devanagari digits, Latin digits, and spelled-out numbers.
- **Punctuation and casing.** The model produces both; Kaldi-style references have neither.
- **Unicode normalisation.** Composed versus decomposed forms of the same character.
- **Compound splitting** of technical terms into multiple ordinary words.

Define two normalisation levels:

Level 1 — standard, script-preserving. Unicode normalisation, numeral unification, punctuation removal, case folding, whitespace collapse. This produces the **headline WER**, comparable with published baselines on this corpus.

Level 2 — script-invariant. Additionally romanise Devanagari so that a word written in either script compares equal. This is a **clearly labelled secondary metric**, never presented as the WER.

Validation gate. Before accepting any baseline number, print twenty reference/hypothesis pairs side by side and inspect them manually. A baseline WER above roughly 60% on this corpus is far more likely to be a normalisation defect than a model failure.

8.2 Metrics

Primary — decomposed WER. Report WER split by whether the reference word is in the bias lexicon:

- **B-WER**: error rate restricted to words *in* the syllabus term lexicon.
- **U-WER**: error rate restricted to words *not* in the lexicon.

This decomposition tells the entire story in two numbers. Effective grounding lowers B-WER while leaving U-WER unchanged. Over-biasing lowers B-WER *and raises U-WER*. The gating hypothesis (H4) is precisely a claim about achieving the first pattern rather than the second, and cannot be demonstrated without this split.

Supporting metrics.

- Overall WER, and its substitution / insertion / deletion breakdown.
- Character error rate, which is more robust to tokenisation differences.
- Script-invariant WER (Level 2), reported alongside Level 1 to quantify the orthographic share of total error.
- Terminology precision, recall and F1.
- Percentage of utterances improved and percentage regressed relative to baseline.
- Retrieval top-1 topic accuracy.
- Guard firing rates.

8.3 Statistical significance

Mean WER differences on a few hundred utterances are easily noise. Use a **paired bootstrap test**: resample utterances with replacement many times, recompute the aggregate error rate for both systems on each resample, and report the proportion of resamples in which the grounded system wins. Report a p-value for every system against the baseline.

Store **per-utterance edit counts and reference lengths**, not just corpus aggregates — the test requires them, and recomputing from cached hypotheses is otherwise impossible.

8.4 Error analysis

Allocate real time to this; it produces the paper's discussion section.

1. WER as a function of utterance duration.
2. The hundred most frequent substitution pairs, each manually classified as orthographic, terminology, function-word, or hallucination error.
3. The share of total word errors falling on lexicon terms — the **headroom estimate**.

The headroom estimate should be computed and stated *before* presenting results. If terminology accounts for only a small fraction of total errors, the maximum achievable WER gain is correspondingly bounded. Predicting a modest result in advance and then observing it is a far stronger position than reporting a modest result unexplained.

---

9. Experiment Matrix

9.1 Conditions

ID	System	What it isolates
B0	Baseline, no grounding	Reference point
C1	Generic non-syllabus context	Effect of conditioning *per se*
C2	**Randomly chosen** syllabus document	Whether the *correct* syllabus matters
C3	**Oracle** syllabus document	Upper bound given perfect retrieval
M1	Retrieved context conditioning	Mechanism 1
M2	Retrieved token-level biasing	Mechanism 2
M3	Output-level correction on B0	Mechanism 3
M2+M3	Best decode-time plus correction	Complementarity (H3)
G	Confidence-gated biasing	Principal contribution (H4)

C2 and C3 are what make this a study rather than a demonstration. C2 controls for retrieval quality: if a random syllabus document helps as much as the correct one, the gain comes from generic prompting and the central claim fails. C3 establishes the ceiling and separates retrieval failure from biasing failure.

9.2 Compute budget

Not every condition requires a decode. Sorting by real cost:

Condition	Decode cost
B0, C1, C2, C3, M1, M2	one decode each
M3, and every combination with M3	**free** — operates on cached text
G	partial — re-decodes only flagged utterances

The full matrix is therefore approximately six and a fraction decodes, not nine. On Tier 2 this is roughly eight hours of compute: one or two overnight runs. Tier 3 is reserved for the baseline and the single best system — two runs, about twelve hours, executed once at the end.

Total project compute is on the order of twenty hours, spread across a few unattended overnight sessions.

Execution order. Decode B0 first and completely, on every tier. Every free condition depends on its cached output, and confidence gating requires its token-level confidence scores.

If the budget must be cut, drop C1 first: C2 is a stronger control covering overlapping ground. The minimum viable matrix is B0, C2, C3, M1, M2, M3 and G.

9.3 Batched inference

The runtime provides a batched inference mode with a configurable batch size, which gives a substantial speedup over sequential decoding on CPU. Benchmark batched against sequential on a small sample before committing, and set worker thread count to the available core count.

---

10. Threats to Validity

State these explicitly in the paper; anticipating them is stronger than being asked.

1. **Weakened baseline.** If the turbo model underperforms on this language pair, gains may partly reflect recovery of a self-inflicted deficit. Mitigated by the Section 4.2 pilot and, ideally, by confirming the headline result on the larger model.
2. **Test-set leakage through the lexicon.** Mitigated by constructing the syllabus independently of test transcripts and freezing the lexicon before any grounded run.
3. **Tuning on evaluation data.** Mitigated by the disjoint Tier 1 development slice.
4. **Metric dependence on lexicon definition.** B-WER and U-WER are defined by lexicon membership; a differently drawn lexicon yields different numbers. Mitigated by freezing and fully documenting the lexicon, and by reporting overall WER alongside.
5. **Subset sampling.** Ablations run on a sample rather than the full test set. Mitigated by a recorded seed and by final

confirmation on the complete test set.

6. **Single corpus, single language pair.** Generalisation to other code-switched pairs is untested. State as a limitation; the corpus contains a Bengali–English portion that a future study could use.
7. **Non-determinism and version drift.** Fix random seeds, temperature zero, and record model, runtime and library versions.

---

11. Milestones

Stage	Deliverable	Gate to pass before proceeding
1	Environment, corpus downloaded, manifest built, tiers frozen	Counts and duration match published figures
2	Model running locally; caching verified; model and language pilots complete	Pilot results recorded; configuration fixed
3	Normalisation and scoring harness; baseline B0 locked	Twenty pairs manually inspected; baseline WER plausible
4	Error analysis; headroom estimate; substitution table	Headroom figure stated in writing
5	Syllabus documents authored; chunk index and lexicon built and **frozen**	Lexicon coverage statistic recorded
6	Retrieval implemented; retrieval accuracy measured	Top-1 topic accuracy reported
7	M1 and M2 implemented with guards	Guard firing rates logged
8	M3a and M3b; thresholds swept on Tier 1	Chosen values and sweep curves recorded
9	Confidence gating (G); threshold sweep; trade-off frontier plotted	Frontier plot produced
10	Full matrix on Tier 2; bootstrap tests	p-values for all systems against B0
11	B0 and best system on Tier 3	Final table complete
12	Figures, paper draft, transcripts handed to note-generation stage	—

---

12. Paper Structure and Experiment Mapping

Paper section	Supplied by
Introduction and motivation	Sections 2.1–2.3
Related work	Contextual biasing, code-switched ASR, Whisper adaptation
Dataset	Section 3, plus corpus statistics
Method: architecture	Section 6 two-pass pipeline figure
Method: mechanisms	Section 7, M1–M3 and G
Experimental setup	Section 4 pilots; Section 3.3 tiers; Section 8.1 normalisation
Results: main table	Section 9.1 matrix with WER, B-WER, U-WER, p-values
Results: controls	C2 and C3, establishing that syllabus content matters
Results: gating	G trade-off frontier, addressing H4
Analysis	Section 8.4 error taxonomy; orthographic decomposition
Limitations	Section 10
Conclusion and future work	Retrieval ceiling from C3; Bengali–English extension; fine-tuning comparison

Candidate title. *Syllabus-Grounded Contextual Biasing for Code-Switched Lecture ASR: A Comparison of Injection Mechanisms.*

Venue calibration. Confirm the target venue with the project guide before finalising scope. A departmental or student-track venue accepts a rigorous evaluation study of this form. A top-tier speech venue would additionally expect a comparison against parameter- efficient fine-tuning on the corpus train split, which requires GPU access and roughly three additional weeks — a decision to take early, not late.

---

13. Reproducibility Checklist

Confirm each item before submission.

- Corpus version, subset seeds and tier sizes recorded.
- Model identifier, quantisation setting, beam width, temperature and language setting recorded.
- Runtime and library versions pinned.
- Syllabus documents and frozen term lexicon included as supplementary material.
- Retrieval model identifier, chunk size, overlap and top-k recorded.
- All decode outputs cached and archived.
- Per-utterance edit counts retained for significance testing.
- Normalisation procedure fully specified, both levels.
- Guard thresholds and firing rates reported.
- Every hyperparameter selected on the development tier, stated as such.